



Grafana

Modelo de Uso



Versão 1.0

30/12/2021

TODAS AS INFORMAÇÕES AQUI CONTIDAS SÃO CONFIDENCIAIS.

Esse nível é atribuído às informações cujo conhecimento é do interesse de toda instituição, incluindo empregados CAIXA, prestadores de serviço, usuário Fábrica e menores aprendizes no exercício de suas atribuições. São informações necessárias ao desempenho de suas atribuições na CAIXA, não cabendo o seu conhecimento a empregados de empresas contratadas que não possuam nenhuma atribuição junto ao serviço ou produto contratado. Outras pessoas ou empresas deverão ser autorizadas pela **SUCTI**, de acordo com a política vigente sobre divulgação de informações corporativas.

Este documento contém informações proprietárias.

HISTÓRICO DE REVISÕES

Data	Versão	Descrição	Autor
30/12/2021	1.0	Criação do documento.	Manuel Nunes Pereira

Sumário

1.	Glossário	5
2.	Introdução	6
3.	Arquitetura	7
3.1.	Modelo Push x Modelo Pull	8
4.	Prometheus	9
4.1.	Instrumentar a aplicação	10
4.2.	Métricas do Prometheus	11
4.3.	Tipos de métricas	13
5.	Consultas com o PromQL	14
5.1.	Tipos de dados	14
5.2.	Filtro de dados	16
5.3.	Funções	17
5.4.	Consulta de counters	19
6.	Grafana	20
6.1.	Estrutura do Grafana	21
6.1.1.	Data Sources	21
6.1.2.	dashboard	21
6.1.3.	Interface do dashboard	21
6.1.4.	Cabeçalho do dashboard	22
6.1.5.	Painéis de visualização	23
6.2.	Consultas no Grafana	26
6.2.1.	Linhas de consulta	26
6.3.	Configurações do Dashboard	27
6.3.1.	Variáveis	28
6.3.2.	Versões	29
6.3.3.	Permissões	29
6.4.	Modelos de Dashboard	30
6.4.1.	Como criar modelos	30
6.4.2.	Como repetir linhas	34
6.5.	Alertas	36

6.5.1.	Como criar um canal de notificação	38
6.5.2.	Como criar um alerta	39
6.5.3.	Campos de regra de alerta	40
6.5.4.	Condições	41
6.5.5.	Série Múltipla	43
6.5.6.	Notificações	45
7.	Modelo CAIXA	46
7.1.	Monitoração devops	47
7.2.	Organização das pastas	49
7.3.	Principais painéis	50
7.3.1.	INFRA DEVOPS OKD	50
7.3.2.	INFRA DEVOPS VM	51
8.	Suporte	53

1. GLOSSÁRIO

Banco de séries temporais: (*Time series database* - TSDB) é um sistema otimizado para armazenar dados temporais, normalmente em formato *timestamp* com um valor associado.

Grafana: poderosa ferramenta de monitoramento web que fornece tabelas, gráficos e alertas quando conectado as fontes de dados suportadas. É expansível através de um sistema de plug-in.

Instrumentação: Para monitorar seus serviços, é necessário instrumentar o código por meio de uma das bibliotecas clientes do Prometheus que corresponda a linguagem no qual a aplicação foi escrita. Por exemplo, bibliotecas para Java, C++, nodejs, PHP, .net, entre outras.

Monitoramento: ação de acompanhar e visualizar o comportamento da aplicação de maneira objetiva, utilizando ferramentas que permitam criar gráficos, tabelas e alertas (*stack de monitoramento*). Pode-se dividir o monitoramento em dois grupos principais para coleta e análise: métricas de infraestrutura e métricas da aplicação.

Prometheus: ferramenta utilizada para o monitoramento em tempo real de milhares de métricas de aplicações ou infra. O *Prometheus* coleta e armazena métricas como dados de séries temporais (TSDB).

PromQL: linguagem de consulta utilizado pelo *Prometheus* e Grafana que nos possibilita selecionar e agregar dados de séries temporais em tempo real. Esta poderosa linguagem de expressão nos permite realizar filtros utilizando os rótulos de séries temporais e utilizar os resultados de cada expressão para visualizar graficamente.

Scrape: ato de coletar métricas por uma ferramenta coletora, como o *Prometheus*, em um *endpoint* previamente definido.

2. INTRODUÇÃO

Este documento tem como objetivo apresentar as principais características das ferramentas Grafana e *Prometheus*, instrumentos essenciais para o processo de monitoramento dos aplicativos CAIXA, em especial, os desenvolvidos pelas esteiras DevOps.

O conhecimento sobre estas ferramentas e sobre o modelo adotado facilita a compreensão e a democratização do seu uso, tornando-o acessível a todos os interessados, desenvolvedores, operações e suporte, para acompanhamento e monitoração da aplicação publicada, inclusive para ser alertado sobre eventuais problemas.

A primeira parte do documento apresenta detalhes sobre a arquitetura, a lógica de captura de dados, *Prometheus* e PromQL.

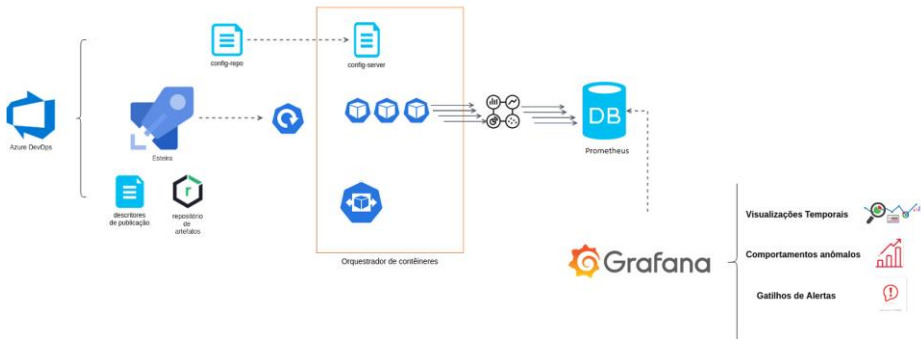
A segunda parte do documento apresenta o Grafana e suas principais características, formas de construir consultas, painéis, variáveis e alertas.

A terceira parte apresenta os principais painéis utilizados em <http://monitoracaodevops.produtos.caixa>, pesquisa e solicitação de suporte.

3. ARQUITETURA

A arquitetura adotada para monitoramento das esteiras de DevOps contempla fundamentalmente o Prometheus, como ferramenta de coleta e armazenamento de dados de séries temporais, e o Grafana, como ferramenta de visualização estruturada desses dados.

Fluxo da arquitetura



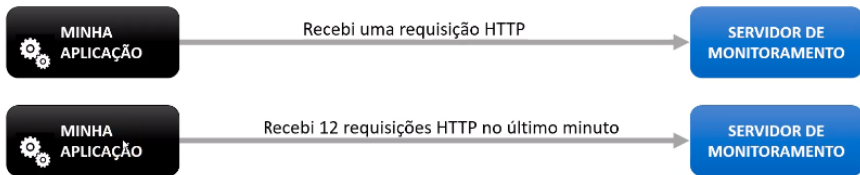
Além de estar apto para gerar dados de monitoramento em qualquer ambiente, o *Prometheus* tem uma versão integrável ao OKD, favorecendo um conjunto pré-definido de bibliotecas para monitoramento em ambientes de *container*.

Além do *Prometheus*, o modelo de monitoramento das esteiras DevOps utiliza outros tipos de data sources, como o *ElasticSearch* e o *InfluxDB*.

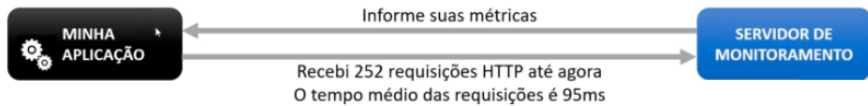
3.1. MODELO PUSH X MODELO PULL

Para realizar a monitoração, existem dois modelos para carga de dados. A aplicação pode enviar os dados para um servidor de monitoramento (modelo push) ou o servidor de monitoramento vai de tempos em tempos até a aplicação e captura o estado dela (modelo pull).

PUSH



PULL



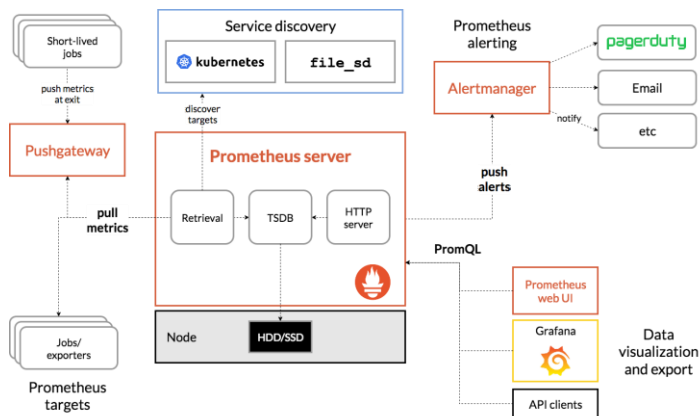
Não há um modelo melhor do que o outro, pois depende de decisões, necessidades e ferramentas utilizadas.

O modelo utilizado para monitoramento das esteiras DevOps é o PULL.

4. PROMETHEUS

A imagem abaixo apresenta a estrutura utilizada pelo *Prometheus*, e seus principais componentes são:

- um servidor que gerencia serviços, recupera e armazena métricas de aplicações monitoradas (retrieval) e realiza análise de dados de séries temporais usando a linguagem de consulta PromQL;
- um database de série temporal (TSDB);
- um servidor http que responde as solicitações da API (HTTP Server);
- um simples built-in GUI e suporte nativo ao Grafana;
- um gerenciador de alertas (alertmanager), no qual os alertas podem ser definidos na linguagem de consulta e;
- um gateway de envio (Pushgateway), útil para monitorar aplicações de curta duração.



4.1. INSTRUMENTAR A APLICAÇÃO

O *Prometheus* dá suporte ao uso de uma combinação de instrumentação e agentes, os chamados exportadores.

Para instrumentação é necessário acesso ao código-fonte da aplicação para inclusão de scripts (bibliotecas) e com isso possibilitar a captura de métricas personalizadas.

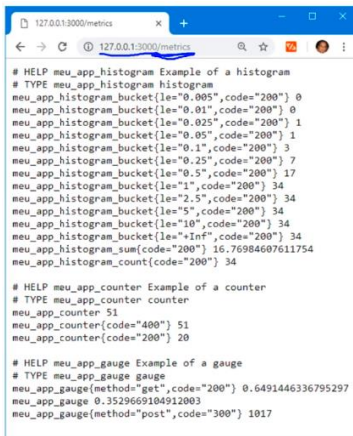
O *Prometheus* é **agnóstico** em linguagem de programação e possui bibliotecas que são compatíveis e que estão disponíveis para Go, Java / Scala, Python e Ruby. Muitas bibliotecas não oficiais também estão disponíveis (LISP, .net, PHP, etc.) sendo possível ainda desenvolver suas próprias bibliotecas.

Há cenários que não é viável instrumentar códigos de um determinado sistema diretamente com as métricas do *Prometheus*, como no caso de sistemas operacionais, como Linux ou Windows. Neste caso, deve-se utilizar os exportadores, bibliotecas e servidores que ajudam a **exportar** métricas existentes.

O site oficial do *Prometheus* disponibiliza uma série de bibliotecas que atuam como exportadores: <https://prometheus.io/docs/instrumenting/exporters/>

4.2. MÉTRICAS DO PROMETHEUS

O *Prometheus* atua com o modelo PULL, ou seja, deve-se configurar o Prometheus para capturar determinadas métricas (scraping) em URLs específicas (por padrão ele usa o seguinte *endpoint*: *http://<aplicação>/metrics*) que passará a monitorar a aplicação conforme formatos e consultas definidas.



```
# HELP meu_app_histogram Example of a histogram
# TYPE meu_app_histogram histogram
meu_app_histogram_bucket{le="0.005",code="200"} 0
meu_app_histogram_bucket{le="0.01",code="200"} 0
meu_app_histogram_bucket{le="0.025",code="200"} 1
meu_app_histogram_bucket{le="0.05",code="200"} 1
meu_app_histogram_bucket{le="0.1",code="200"} 3
meu_app_histogram_bucket{le="0.25",code="200"} 7
meu_app_histogram_bucket{le="0.5",code="200"} 17
meu_app_histogram_bucket{le="1",code="200"} 34
meu_app_histogram_bucket{le="2.5",code="200"} 34
meu_app_histogram_bucket{le="5",code="200"} 34
meu_app_histogram_bucket{le="10",code="200"} 34
meu_app_histogram_bucket{le="+Inf",code="200"} 34
meu_app_histogram_sum{code="200"} 16.76984607611754
meu_app_histogram_count{code="200"} 34

# HELP meu_app_counter Example of a counter
# TYPE meu_app_counter counter
meu_app_counter 51
meu_app_counter{code="400"} 51
meu_app_counter{code="200"} 20

# HELP meu_app_gauge Example of a gauge
# TYPE meu_app_gauge gauge
meu_app_gauge{method="get",code="200"} 0.6491446336795297
meu_app_gauge 0.3529669104912003
meu_app_gauge{method="post",code="300"} 1017
```

No exemplo acima, cada linha é preenchida por uma métrica definida e suas variações (rótulos distintos). No exemplo, há três diferentes tipos de métricas apresentadas: *app_histogram_bucket*, *app_counter* e *app_gauge*.

Por exemplo: `http_request_total {method="post", code="200"} 27`

Neste exemplo, a **métrica** é "`http_request_total`", ou seja, o total de requisições via http, e dois **rótulos** distintos, um que identifica o método utilizado "`post`" e outro que identifica o código de retorno "`200`" e, finalmente, o **valor** obtido pela incidência da métrica (27).

Observe que o rótulo permite obter variações da métrica. Se o rótulo fosse diferente, como o código de resposta 500, o valor obtido seria diferente.

E como o *Prometheus* armazena as series temporais?

Para cada combinação de métrica e rótulo, há uma "tabela" com os dados coletados.

No exemplo abaixo, há três variações da métrica `http_request_total`, variando apenas o rótulo do código de retorno (`code`).

Neste caso, há três “tabelas” de séries temporais, variando conforme o rótulo e armazenando para cada uma a quantidade de observações e um *timestamp* (para indicar a referência temporal).

No final temos a métrica (com um rótulo) e o valor da incidência (dados na tabela)

- `http_request_total {method="post", code="200"} 0`
- `http_request_total {method="post", code="404"} 1`
- `http_request_total {method="post", code="500"} 5`

É importante ser econômico e evitar muitas variações para cada métrica para não criar tabelas desnecessárias que irão gerar dados a serem coletados e armazenados.

4.3. TIPOS DE MÉTRICAS

É possível optar por quatro tipos principais de métricas no *Prometheus*:

- counter (contador)
- gauge (medidor)
- histogram (histograma)
- summary (resumo)

Um **counter** ou **contador** é uma métrica incremental cumulativa cujo valor só pode aumentar ou ser zerado na reinicialização. Por exemplo, pode-se usar um contador para representar o número de solicitações atendidas, tarefas concluídas ou erros.

Um **gauge** ou **medidor** é uma métrica indicada para valores que podem aumentar ou diminuir arbitrariamente. Os medidores são normalmente usados para valores medidos como temperaturas ou espaço em disco, mas também "contagens" que podem aumentar e diminuir, como o número de solicitações simultâneas.



Um **histograma** é uma métrica que mostra valores como tempo de resposta e as apresentam em intervalos configuráveis. Esta métrica é recomendada quando a média é uma métrica muito limitada para aquilo que se quer observar. Ex.: tempo de

resposta de uma aplicação com quatro intervalos, o primeiro com valores menores que 100 milissegundos (ms), o segundo com 101 a 200 ms, o terceiro com 201 a 500 ms e o último com valores superior a 500 ms.



Um **summary** ou **resumo** é uma métrica semelhante a um *histograma* e embora forneça uma **contagem total** de observações, ela calcula a distribuição destes em percentuais configuráveis e os apresenta em buckets (agrupamento de dados).

5. CONSULTAS COM O PROMQL

Uma consulta apresenta a seguinte estrutura:

- O **nome** da métrica;
- **rótulos**, ou seja, pares de valores-chave para distinguir métricas com o mesmo nome;
- o **valor métrico**, um número de ponto flutuante de 64 bits;
- um **timestamp** com precisão de nível de milissegundo.

Ex: `prometheus_http_requests_total {method="post", code="200"} 50`

5.1. TIPOS DE DADOS

Os dados podem ser dos seguintes tipo:

- scalar
- instant vector
- range vector

scalar é um vetor de séries temporais. Por exemplo, a consulta `api_requests_total` pode trazer as seguintes séries:

```
prometheus_http_requests_total {method="post", code="200"}  
prometheus_http_requests_total {method="post", code="404"}  
prometheus_http_requests_total {method="post", code="500"}
```

Todas apresentam o mesmo método e variam conforme o rótulo de *http code*. Portanto, o vetor dessas séries temporais é o `<code>`.

instant vector é um vetor em um determinado timestamp. Por exemplo, ao realizar uma consulta com o seguinte timestamp `2021-11-26 03:10:00`, temos os seguintes resultados:

```
prometheus_http_requests_total {method="post", code="200"} 367  
prometheus_http_requests_total {method="post", code="404"} 21  
prometheus_http_requests_total {method="post", code="500"} 6
```

range vector são vetores obtidos em um determinado intervalo de tempo. Por exemplo, ao realizar a consulta `prometheus_http_requests_total [1m]`, o timestamp informado (2021-11-26 03:10:00) representa o fim do intervalo.

Se considerar um scrape a cada 15 segundos, o resultado será:

Série Temporal	2021-11-26 03:09:15	2021-11-26 03:09:30	2021-11-26 03:09:45	2021-11-26 03:10:00
<code>prometheus_http_requests_total (method="post", code="200")</code>	360	363	365	367
<code>prometheus_http_requests_total (method="post", code="404")</code>	18	20	21	21
<code>prometheus_http_requests_total (method="post", code="500")</code>	4	5	5	6

5.2. FILTRO DE DADOS

Consultas podem ser filtradas, especialmente por rótulos, que devem ser incluídas por {chaves}. Por exemplo, a consulta `prometheus_http_requests_total` pode ser filtrada pelo tipo de código de retorno 200. Por exemplo, ao se utilizar a interface do prometheus, a sintaxe seria: `prometheus_http_requests_total{code="200"}`

☒ Use local time ☐ Enable query history ☐ Enable autocomplete ☐ Enable highlighting ☐ Enable linter

Q `prometheus_http_requests_total{code="200"}` Executar

Table Graph

Load time: 31ms Resolution: 14s Result series: 11

	Evaluation time
<code>prometheus_http_requests_total{code="200", handler="/ready", instance="localhost:9090", job="prometheus"}</code>	2
<code>prometheus_http_requests_total{code="200", handler="/api/v1/label/name/values", instance="localhost:9090", job="prometheus"}</code>	4
<code>prometheus_http_requests_total{code="200", handler="/api/v1/query", instance="localhost:9090", job="prometheus"}</code>	26
<code>prometheus_http_requests_total{code="200", handler="/api/v1/query_range", instance="localhost:9090", job="prometheus"}</code>	1
<code>prometheus_http_requests_total{code="200", handler="/api/v1/series", instance="localhost:9090", job="prometheus"}</code>	1
<code>prometheus_http_requests_total{code="200", handler="/classic/graph", instance="localhost:9090", job="prometheus"}</code>	2
<code>prometheus_http_requests_total{code="200", handler="/classic/static/filepath", instance="localhost:9090", job="prometheus"}</code>	24
<code>prometheus_http_requests_total{code="200", handler="/graph", instance="localhost:9090", job="prometheus"}</code>	2
<code>prometheus_http_requests_total{code="200", handler="/manifest.json", instance="localhost:9090", job="prometheus"}</code>	1
<code>prometheus_http_requests_total{code="200", handler="/metrics", instance="localhost:9090", job="prometheus"}</code>	16
<code>prometheus_http_requests_total{code="200", handler="/static/filepath", instance="localhost:9090", job="prometheus"}</code>	5

Add Panel Remove Panel

Curiosidades sobre *http codes*:

- Respostas de informação (100-199),
- Respostas de sucesso (200-299),
- Redirecionamentos (300-399)
- Erros do cliente (400-499)
- Erros do servidor (500-599).

5.3. FUNÇÕES

Uma função nada mais é do que acrescentar uma lógica numa consulta de tal forma a calcular um valor que atenda a uma determinada necessidade.

Exemplos de funções:

delta(v range-vector): calcula a diferença entre o primeiro e o último valor de cada elemento da série temporal em um vetor de intervalo v, retornando um vetor instantâneo com os deltas fornecidos e rótulos equivalentes. O delta é extrapolado para cobrir todo o intervalo de tempo, conforme especificado no seletor de vetor de intervalo, de modo que seja possível obter um resultado não inteiro, mesmo se os valores de amostra forem todos inteiros.

O exemplo de expressão a seguir retorna a diferença na temperatura da CPU entre agora e 2 horas atrás:

```
delta(cpu_temp_celsius{host="zeus"}[2h])
```

delta só deve ser usado com medidores.

histogram_quantile(ϕ scalar, b instant-vector): calcula o quantil ϕ ($0 \leq \phi \leq 1$) dos intervalos b de um histograma. (Veja histogramas e resumos para uma explicação detalhada de ϕ -quantis e o uso do tipo de métrica de histograma em geral.) As amostras em b são as contagens de observações em cada intervalo. Cada amostra deve ter um rótulo leon de o valor do rótulo denota o limite superior inclusivo do intervalo. (Amostras sem esse rótulo são silenciosamente ignoradas.) O tipo de métrica de histograma fornece automaticamente a série temporal com o _bucketsufixo e os rótulos apropriados.

Use a rate()função para especificar a janela de tempo para o cálculo do quantil.

Exemplo: uma métrica de histograma é chamada http_request_duration_seconds. Para calcular o 90º percentil das durações das solicitações nos últimos 10 m, use a seguinte expressão:

```
histogram_quantile(0.9, rate(http_request_duration_seconds_bucket [10m]))
```

increase(v range-vector): calcula o aumento na série temporal no vetor de intervalo. As quebras na monotonicidade (como redefinições do contador devido a reinicializações do alvo) são ajustadas automaticamente. O aumento é extrapolado para cobrir todo o intervalo de tempo conforme especificado no seletor de vetor de intervalo, de modo que seja possível obter um resultado não inteiro mesmo se um contador aumentar apenas em incrementos de inteiros.

O exemplo de expressão a seguir retorna o número de solicitações HTTP medidas nos últimos 5 minutos, por série temporal no vetor de intervalo:

```
increase(http_requests_total{job="api-server"}[5m])
```

rate(v range-vector): calcula a taxa média de aumento por segundo da série temporal no vetor de intervalo. As quebras na monotonicidade (como redefinições do contador devido a reinicializações do alvo) são ajustadas automaticamente. Além disso, o cálculo extrapola para o final do intervalo de tempo, permitindo scrapes perdidos ou alinhamento imperfeito de ciclos de scrape com o período de tempo do intervalo.

O exemplo de expressão a seguir retorna a taxa por segundo de solicitações HTTP, medida nos últimos 5 minutos, por série temporal no vetor de intervalo:

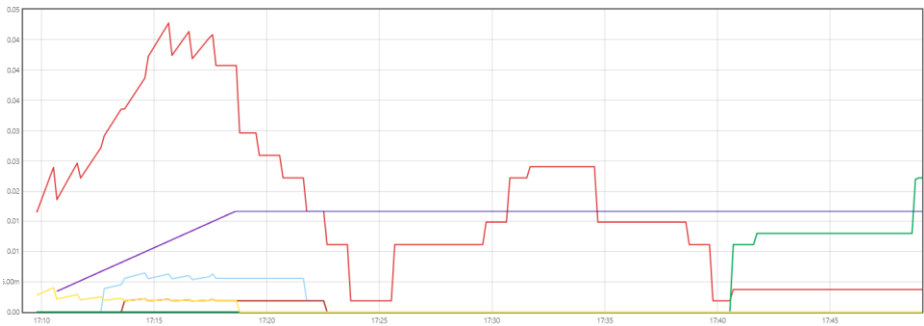
```
rate(http_requests_total{job="api-server"}[5m])
```

A função `rate` só deve ser usada com contadores. É mais adequada para alertas e gráficos de contadores que se movem lentamente.

5.4. CONSULTA DE COUNTERS

As consultas realizadas com este tipo de métrica é incremental e nos traz curvas sempre ascendentes e não nos dá a visibilidade do valor incremental em determinado período. O que fazer para observar um comportamento incremental? Neste caso, aplica-se a função **rate** em um determinado **range vector** (ex: 15 minutos) e um gráfico irá apresentar os valores incrementais.

Ex: `rate(prometheus_http_requests_total{code="200"}[15m])`



6. GRAFANA

O Grafana é uma ferramenta que adota uma poderosa interface gráfica para fornecer informações relevantes do comportamento de aplicação e infra, onde quer que estejam e visualizá-los através de consultas no formato que desejar.

O Grafana foi construído com base no princípio de que os dados devem ser acessíveis a todos na organização, não apenas a grupos específicos, como de operações. Ao democratizar o acesso aos dados e visualizações, o Grafana ajuda a criar uma cultura onde os dados podem ser facilmente acessados por pessoas que precisam deles, ajudando a monitorar comportamentos, resolver tempestivamente problemas, desconstruir silos e promover a capacitação de equipes.

Outro aspecto importante é poder compartilhar os painéis criados com outros membros da equipe, permitindo que explorem os dados de maneira concomitante, promovendo a colaboração e transparência.

6.1. ESTRUTURA DO GRAFANA

6.1.1. DATA SOURCES

O Grafana pode ser configurado para acessar diversos tipos diferentes de base de dados como o Prometheus, Elasticsearch, InfluxDB, Graphite, OpenTSDB, MySQL, PostgreSQL, Microsoft SQL Server e AWS Cloudwatch, entre outros.

6.1.2. DASHBOARD

Um dashboard ou visualização é uma organização customizada de um ou mais gráficos. O Grafana possui uma grande variedade de gráficos a serem incluídos e customizados, conforme a consulta ao data source desejado.

6.1.3. INTERFACE DO DASHBOARD

A interface do painel apresenta a seguinte estrutura:



- 1) o intervalo de tempo de observação;
- 2) uma lista com seletor de tempo. Aqui você pode acessar as opções de intervalo de tempo relativo, opções de atualização automática e definir intervalos de tempo absolutos personalizados.
- 3) Botão de atualização manual. Irá fazer com que todos os painéis sejam atualizados (buscar novos dados).
- 4) Painel de controle e apresentação. Clique no título do painel para editá-los.

- 5) Legenda do gráfico. Você pode alterar as cores da série, o eixo y e a visibilidade da série diretamente na legenda.

6.1.4. CABEÇALHO DO DASHBOARD



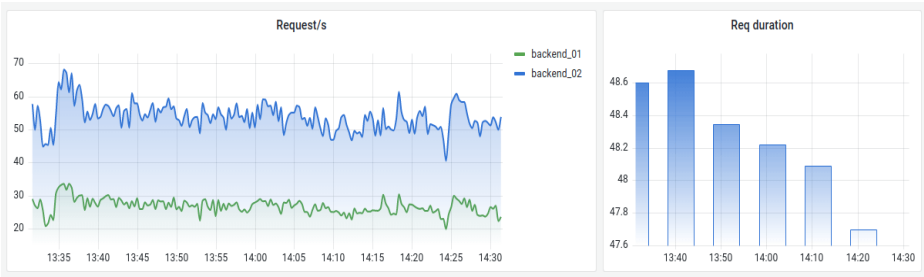
A imagem acima mostra o cabeçalho superior de um painel.

- 1) O menu lateral fornece acesso a recursos não relacionados a um painel, como usuários, organizações e fontes de dados.
- 2) Lista suspensa de painéis: mostra qual painel está sendo visualizado no momento e permite que você alterne facilmente para um novo dashboard. É possível também criar um novo dashboard, importar painéis existentes e gerenciar a lista de painéis.
- 3) Adicionar painel: adiciona um novo painel ao painel atual.
- 4) Favoritos: É possível selecionar o painel atual como favorito. Os painéis favoritos aparecerão em seu painel inicial por padrão e são uma maneira conveniente de marcar os painéis de seu interesse.
- 5) Compartilhar dashboard: compartilhe o painel atual criando um link ou crie um instantâneo estático dele. Certifique-se de que ele seja salvo antes de compartilhar.
- 6) Salvar dashboard: O painel atual será salvo com o nome do painel atual.
- 7) Configurações: gerencie as configurações e recursos do dashboard, como variáveis e anotações.

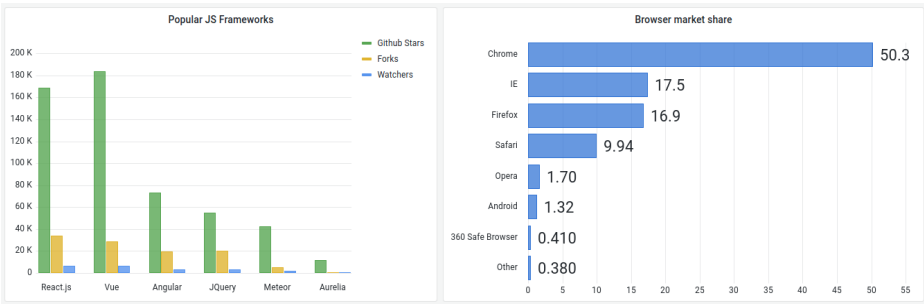
6.1.5. PAINÉIS DE VISUALIZAÇÃO

O Grafana oferece uma grande variedade de visualizações para dar suporte a diferentes necessidades de uso. Abaixo, exemplos de usos mais comuns e suas possibilidades de visualização.

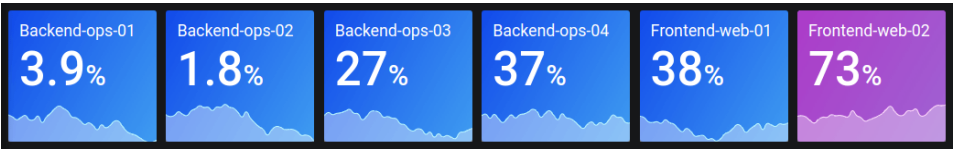
Para gráficos de linha, área e barra com base no tempo, é recomendado a visualização de série(s) temporal(is) padrão.



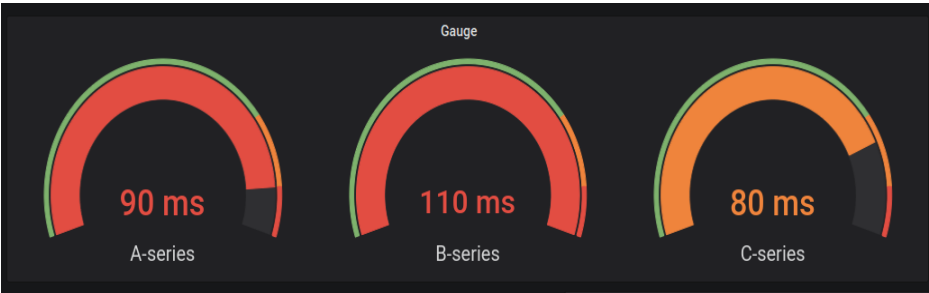
Para dados apresentados em categorias, o ideal é o uso de visualização por gráfico de barras.



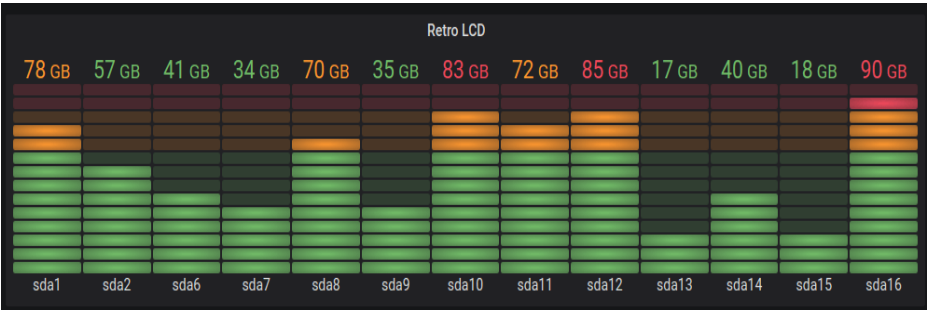
A visualização de estatísticas pode ser apresentada em minigráfico, onde pode-se controlar o fundo ou a cor do valor usando cores distintas.



Para apresentar valores variantes em um intervalo de máximo e mínimo (gauge), pode-se utilizar um medidor radial padrão.



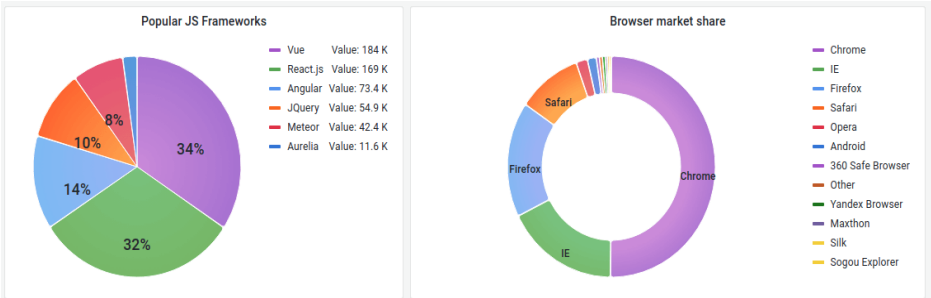
Uma outra opção é utilizar um [medidor de barra](#) horizontal ou vertical com três modos de exibição distintos.



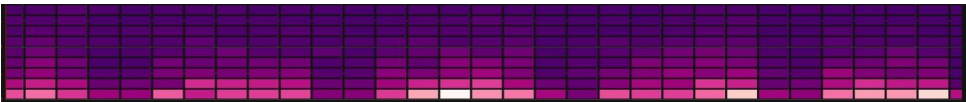
Para mostrar dados detalhados, pode-se utilizar uma visualização de Tabela .

Bar gauge cell display mode					
Time	Info	Min	Max	Value	
2020-09-15 12:45:11	down	73.6 °	76.5 °		74.0 °
2020-09-15 12:39:56	up	73.1 °	76.5 °		75.1 °
2020-09-15 12:27:41	down	72.9 °	76.5 °		74.2 °
2020-09-15 12:40:11	up	73.2 °	76.6 °		75.2 °
2020-09-15 12:27:26	up	73.9 °	76.6 °		74.2 °
2020-09-15 12:44:56	up	72.9 °	76.6 °		74.2 °
2020-09-15 12:39:26	up	72.7 °	76.6 °		74.7 °
2020-09-15 12:42:41	down	73.1 °	76.7 °		74.4 °
2020-09-15 12:51:41	down	73.0 °	76.7 °		75.4 °
2020-09-15 12:41:56	down fast	74.5 °	76.7 °		74.8 °

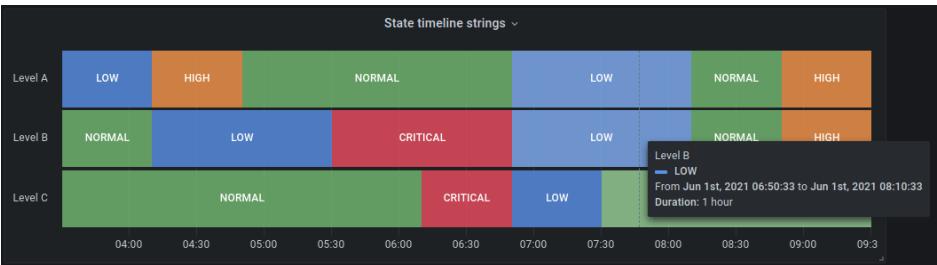
Para dados relativos (percentuais), uma boa maneira de apresentar é utilizando um gráfico de pizza.



Para mostrar a distribuição de valor ao longo do tempo, pode-se usar mapas de calor.



Uma poderosa forma de visualização é o painel de linha do tempo que demonstra mudanças de estado ao longo do tempo. Quando usados com séries temporais, os limites são usados para transformar os valores numéricos em regiões de estado discretas.

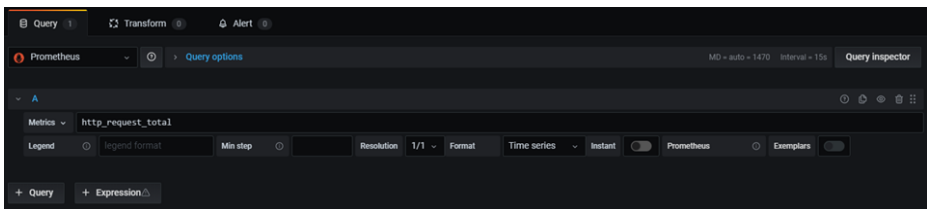


É possível adicionar novos tipos de visualização instalando plug-ins do painel do painel.

6.2. CONSULTAS NO GRAFANA

Uma consulta é a forma como os painéis do Grafana se comunicam com um *data source* selecionado para obter dados e construir uma visualização.

A frequência com que a consulta é enviada para a fonte de dados e quantos dados são coletados podem ser configurados no painel de visualização.



As *data sources* usam diferentes linguagens de consulta e sintaxes para solicitar os dados. Aqui estão dois exemplos de consulta:

PromQL

```
query_result(max_over_time(<metric>[${__range_s}s]) != <state>)
```

PostgreSQL

```
SELECT hostname FROM host WHERE region IN($region)
```

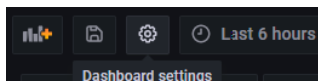
6.2.1. LINHAS DE CONSULTA

Painéis podem ser organizadas em **linhas de consulta**. Cada linha de consulta representa uma consulta que será agregada a visualização e é identificada por uma letra (A, B, C e assim por diante).

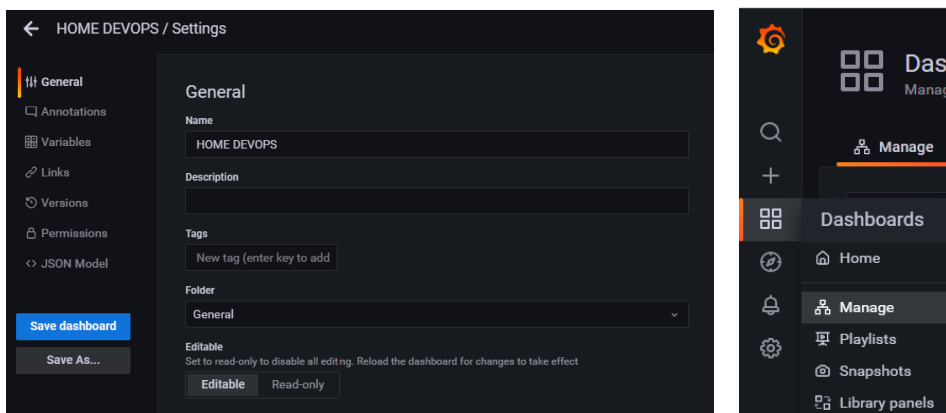
Portanto, o mesmo painel pode apresentar mais de uma consulta simultaneamente.

6.3. CONFIGURAÇÕES DO DASHBOARD

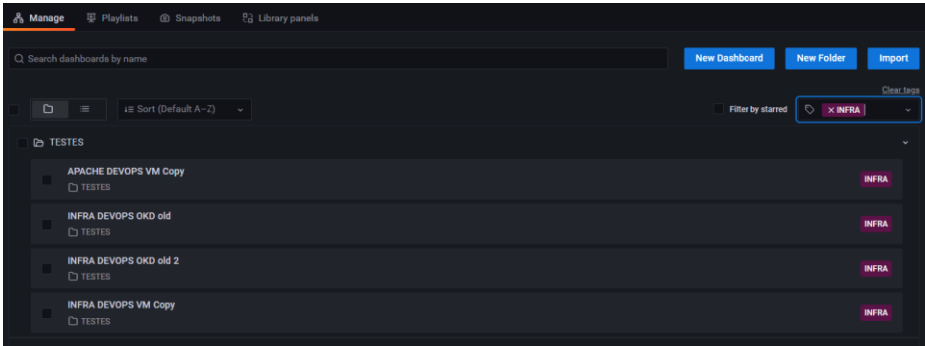
Os dashboards podem ser configurados conforme as necessidades. Para configurar selecione em “dashboard settings”



Uma nova guia será aberta para configurar as informações gerais, como definição do nome, descrição, vinculação de tags e a pasta de organização, além de anotações, variáveis a serem aplicadas e permissões. Painéis podem ser gerenciados na opção “Manage” que fica no menu principal (por padrão, à esquerda).

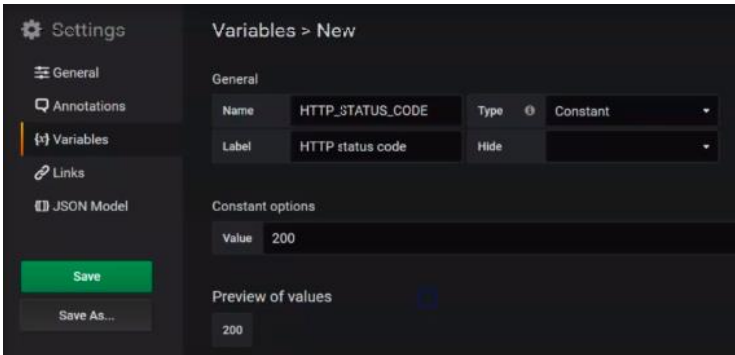


Uma guia será apresentada para gerenciar os painéis criados. Eles podem ser organizados em pastas e podem usar palavras-chave ou tags para facilitar a busca ou filtro.



6.3.1. VARIÁVEIS

É possível trabalhar com variáveis para parametrizar em lote um determinado rótulo.



Ao incluir o nome da variável “http_status_code” na consulta, ele assumirá, neste caso, o valor 200 (neste caso, o rótulo (label) do http code). Se houver interesse, pode-se alterar o valor para uma nova constante, como o valor 500. Neste caso, todas as consultas do painel que inclui esse nome passam a apresentar os resultados para o valor 500 (http code=500).

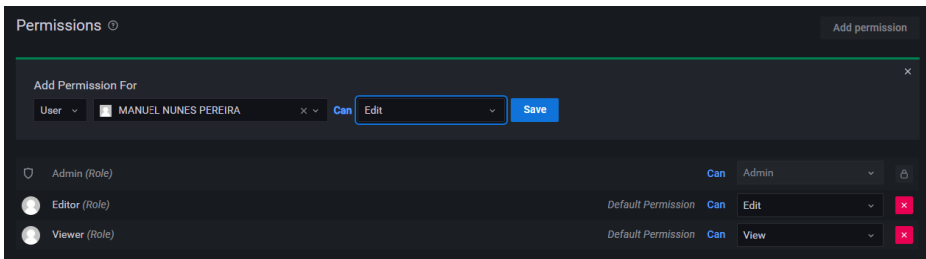
Variáveis são tipificadas (há diversos tipos), e conforme os tipos, mudam-se as configurações. No exemplo acima, o tipo é constante.

6.3.2. VERSÕES

A cada alteração da configuração do dashboard, surge a opção de salvar e uma tela para informar o motivo da mudança (save changes) do dashboard, similar à mensagem de um novo commit. A cada novo “commit”, uma nova versão.

6.3.3. PERMISSÕES

É possível adicionar usuários e dar níveis de permissão (view, edit, admin) em cada dashboard.



Permissions ⓘ

Add permission

Add Permission For

User ▾ MANUEL NUNES PEREIRA x ▾ Can Edit ▾ Save

Admin (Role)	Can	Admin ▾	🔒
Editor (Role)	Default Permission Can	Edit ▾	✖
Viewer (Role)	Default Permission Can	View ▾	✖

6.4. MODELOS DE DASHBOARD

Monitorar diversos aplicativos em execução em diversos ambientes significa que sua plataforma de monitoramento precisa responder dinamicamente ao seu escopo de monitoração. Como visto, variáveis permitem que você reutilize um único painel para todos os aplicativos ou serviços que deseja monitorar.

Portanto, ao selecionar uma variável do seu dashboard, todas as métricas apresentadas relacionadas irão variar, pois são referentes ao novo escopo escolhido, ainda que a estrutura de painel com seus painéis e consultas seja a mesma.

Este conceito de alterar uma variável e manter o mesmo dashboard, faz com que o painel utilizado seja um padrão de visualização ou modelo. É possível criar diversos painéis e adotá-los como novos modelos.

6.4.1. COMO CRIAR MODELOS

Se você tiver permissão e pelo menos uma variável definida para o seu dashboard, então pode configurar um painel para *repetir* para cada valor selecionado nessa variável. Quando os valores na variável mudam, o mesmo acontece com o seu painel.

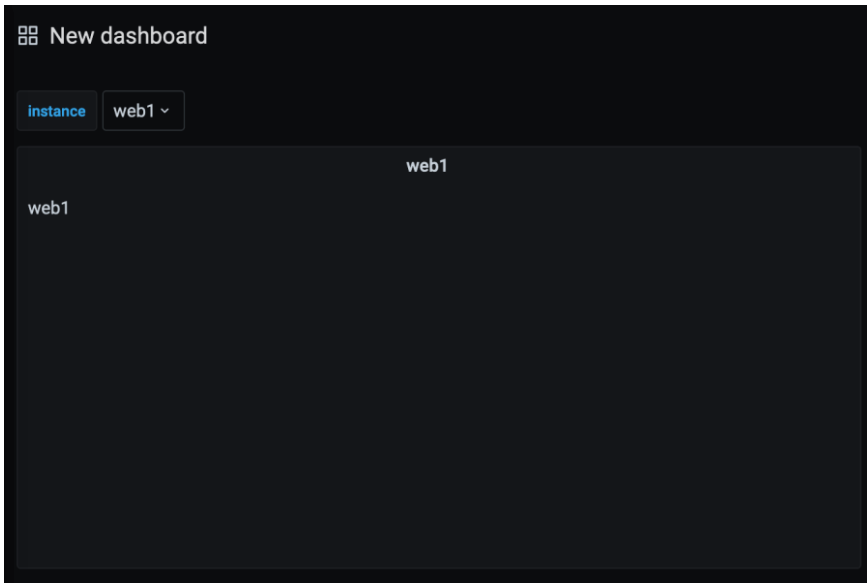
Para habilitar painéis repetidos, no entanto, você precisa criar uma variável.

1. Crie um painel em branco (add panel & save)
2. Vá para as **configurações do painel** clicando no ícone de “engrenagem” no canto superior direito.
3. Vá para a guia **variáveis** e clique em **adicionar variável**

4. Em **nome**, insira **instancia**. É assim que você irá se referir em suas consultas.
5. Em **tipo**, selecione **personalizado (custom)**. As variáveis personalizadas nos permitem adicionar valores manualmente as nossas variáveis, o que será útil para teste.
6. Em **valores separados por vírgula**, digite `web1, web2, web3, backend1, backend2`.
7. Em **opções de seleção**, ative a opção: **multivalorada (multi-value)** e **incluir tudo (include all option)**.
8. Salve (commit) o dashboard.
9. Clique na seta no canto superior esquerdo para voltar ao painel. No canto superior esquerdo do painel, agora há um menu suspenso que contém os valores que você acabou de definir.

Pronto. A variável multivalorada foi criada. A seguir, deve-se criar um painel que usa a variável que criada.

1. Adicione um painel e selecione a visualização de **texto**.
2. Selecione o modo markdown e substitua o texto de amostra por `$instancia`.
3. Após aplicar, configure o título do painel para `$instancia`. Isso define o título de cada painel repetido com o nome da instância.



Tente selecionar uma instância diferente no menu suspenso chamado **instancia**. O painel mostra o valor da variável dentro do painel de texto e no título do painel. Se você selecionar vários valores, o painel exibirá algo semelhante `web1 + web2`, dependendo dos valores selecionados.

Ao alterar o valor da variável, você pode alterar dinamicamente o conteúdo do painel. Em seguida, você configurará o painel para repetir para cada instância selecionada.

1. Edite o painel clicando no título do painel e clique em editar.
2. No editor de painel, abra a seção **opções de repetição (repeat options)**.
3. Em **repetir por variável**, selecione a variável de **instancia**.
4. Na **direção de repetição**, selecione **horizontal** para dispor os painéis repetidos horizontalmente.
5. Aplique suas alterações para voltar ao painel.

Agora, conforme altera sua seleção no menu suspenso, os painéis aparecem e desaparecem de acordo. Você pode até selecionar **todos** para criar um painel para cada instância.

Como os painéis repetidos são baseados no painel original, para editar os painéis repetidos você precisa editar o painel original - todos os painéis repetidos são atualizados depois que você recarrega a página. Não importa quantos painéis você tenha, você pode alterar todos eles em um só lugar.

Os painéis repetidos são úteis para evitar a criação manual de painéis. Mas e se você quiser repetir um *conjunto* de painéis para cada valor selecionado?

6.4.2. COMO REPETIR LINHAS

Vimos como podemos criar painéis dinamicamente a partir de variáveis. A seguir, veremos como adicionar dinamicamente linhas inteiras de painéis. As linhas repetidas permitem que você crie grupos inteiros de painéis dinamicamente.

Imagine que você deseja criar um painel RED. Para cada serviço, você gostaria de três painéis exibindo o tráfego, o número de erros e quanto tempo as solicitações estão demorando. Poderíamos criar painéis repetidos para cada uma dessas métricas, mas o ideal é agrupar os painéis para cada serviço.

Vejamos como você pode repetir conjuntos de painéis, usando *linhas repetidas*.

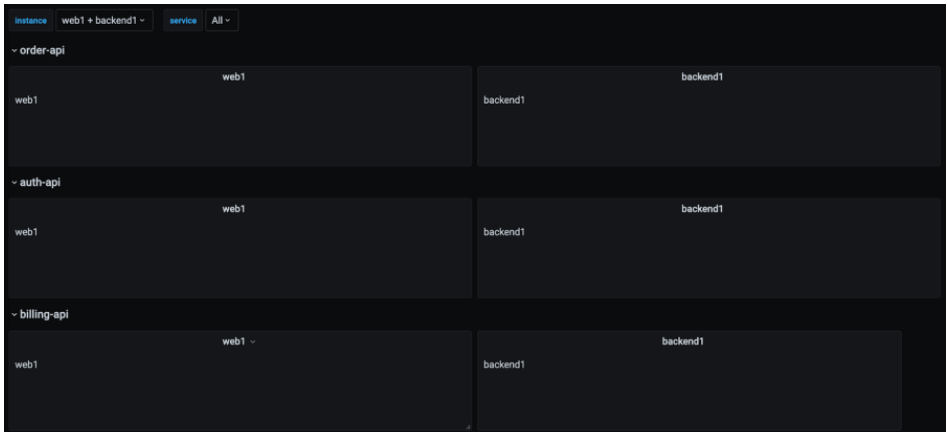
Para este exemplo, vamos criar outra variável personalizada chamada “servico”, contendo order-api, auth-api, billing-api. Siga os mesmos passos de quando você adicionou a variável “instancia”. Não se esqueça de salvar!

Agora, vamos criar uma linha repetida:

1. Clique em adicionar painel e em adicionar em linha (add new row). Os painéis que você criou anteriormente são atribuídos automaticamente à linha que você criou.
2. Passe o cursor sobre o **título da Linha** e clique no ícone de engrenagem para abrir as **opções de linha (row options)**.
3. Em **título**, digite \$servico.
4. Em **repetir para**, selecione a variável para a qual deseja repetir as linhas. Para este exemplo, selecione **servico**.
5. Clique em **atualizar**.

6. Selecione vários serviços no menu suspenso de **serviço**. O Grafana cria uma linha para cada serviço selecionado, cada um dentro de seu próprio conjunto de painéis repetidos.
7. Encontre o painel mais à esquerda na linha mais acima e edite-o.
8. Na área de texto do painel Texto, insira \$instancia and \$serviço.
9. Salve o painel e atualize a página.

Estamos usando o painel Texto aqui para que você possa ver os valores das variáveis de **instância** e **serviço** disponíveis para cada painel repetido. Na prática, porém, você usaria as variáveis em suas consultas para fazer cada painel exibir dados para uma determinada instância e serviço.



6.5. ALERTAS

Os alertas são um aspecto importante do monitoramento porque nos informam de forma proativa quando algo está errado e precisa de nossa atenção. Isto pode acontecer:

- Quando algo trava;
- Quando se está consumindo recursos além do previsto/ideal (por exemplo, memória, CPU);
- Quando há uma interrupção;
- Quando ocorre degradação de desempenho, entre outros...

Alertas bem definidos, estruturados e acionáveis ajudam a identificar e permite propor soluções adequadas aos problemas rapidamente, minimizando a interrupção ou degradação dos serviços.

Ao configurar alertas para o seu sistema, considere o seguinte:

- Deve-se usar alertas apenas quando precisar de intervenção humana;
- Tenha cuidado para não abusar dos alertas. Se alguém receber um alerta com muita frequência e/ou pouco critério, ele pode deixar de ser útil ou servir ao seu propósito.
- Use alertas que sejam diretamente relevantes para o seu cenário. Por exemplo, se você estiver monitorando a infraestrutura, configure alertas para uso de disco, alto uso de CPU ou memória e erros de API.

Existem duas partes de alerta no Grafana: **Regras de Alerta e Canais de Notificação**.

Regras de Alerta: Regras de alerta são a parte mais importante dos alertas Grafana. Regras são condições que você define para quando um alerta é acionado. O Grafana avalia as regras de acordo com um planejador e você precisa especificar a frequência com que as regras são avaliadas.

Em linguagem simples, exemplos de regras podem ser:

- Quando o uso do disco é superior a 90%
- Quando o uso médio de memória é maior que 90% por 5 minutos (este é um exemplo de uma regra baseada em intervalo)
- Quando a temperatura de um dispositivo está fora de uma determinada faixa (este é um exemplo de uma regra com muitas condições diferentes)

Canais de Notificação: Os canais de notificação são para onde os alertas são enviados assim que as regras de alerta são acionadas. Se você não tem canais de notificação, seus alertas só aparecem no Grafana.

Exemplos de canais incluem ferramentas que sua equipe já pode usar:

- caixa-mail
- teams
- telegram, entre outros.

Grafana fornece integração com webhooks, e-mail e mais de uma dúzia de serviços externos. Sempre que criamos um alerta, o atribuímos a um canal de notificação, junto com uma mensagem.

Deve-se pensar nos alertas como objetos que se movem por diferentes estados (máquina de estados), dependendo da regra associada a eles. Os estados possíveis são:

- OK,
- PENDING,
- ALERTING,
- NO DATA.

6.5.1. COMO CRIAR UM CANAL DE NOTIFICAÇÃO

Como mencionado, um alerta no Grafana tem dois componentes - um canal de notificação e uma regra de alerta. Um canal de notificação é definido como a forma de você ser notificado pelo Grafana.

Para criar um canal de notificação, devem ser seguidos os seguintes passos:

- Abra a ferramenta Grafana clicando em Launcher > Primary Region Monitoring.
- No Grafana, clique no menu suspenso no canto superior esquerdo, selecione Alerta > Canais de notificação.
- Nesta tela, crie um novo canal e especifique o método de notificação selecionando o tipo no menu suspenso.

6.5.2. COMO CRIAR UM ALERTA

Há duas coisas importantes para se ter em mente antes de criar um alerta:

- Os alertas não podem ser criados nos painéis de métricas padrão fornecidos com a plataforma.
- Os alertas só podem ser configurados em um painel 'Gráfico' em um painel.
- Alertas e limites só podem ser vistos por usuários com permissão de Editor no Grafana.

Para adicionar ou criar uma regra de alerta:

- Navegue até o painel ao qual deseja adicionar ou editar uma regra de alerta, clique no título e em **Editar**.
- Na guia Alerta, clique em **Criar Alerta**. Se já existir um alerta para este painel, você pode apenas editar os campos na guia Alerta.
- Preencha os campos. As descrições estão listadas abaixo nos campos de regra de alerta .
- Quando terminar de escrever sua regra, clique em **Salvar** no canto superior direito para salvar a regra de alerta e o painel.
- (Opcional, mas recomendado) Clique em **Testar regra** para garantir que a regra retorne os resultados esperados.

Para excluir um alerta

- Para excluir um alerta, role até o final do alerta e clique em **Excluir**.

6.5.3. CAMPOS DE REGRA DE ALERTA

Esta seção descreve os campos que você preenche para criar um alerta.

- **Nome** - digite um nome descritivo. O nome será exibido na lista Regras de alerta. Este campo oferece suporte a modelos .
- **Avaliar a cada** - especifique com que frequência o planejador deve avaliar a regra de alerta. Isso é conhecido como *intervalo de avaliação*.
- **Para** - especifique por quanto tempo a consulta precisa violar os limites configurados antes que a notificação de alerta seja acionada.

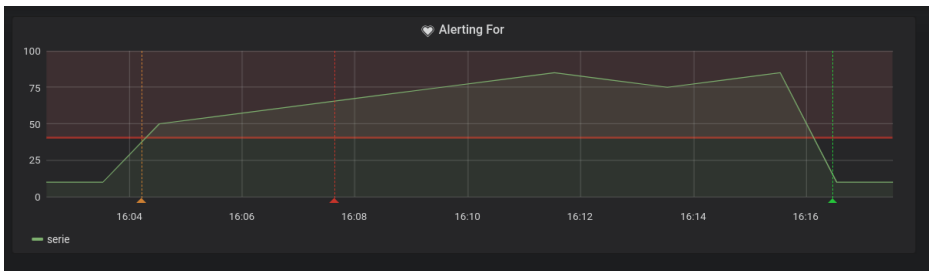
Você pode definir um intervalo mínimo de avaliação no `alerting.min_interval_seconds` campo de configuração, para definir um tempo mínimo entre as avaliações. Consulte Configuração para obter mais informações.

Cuidado: Não use **For** com **If no data or all values are null** configuração definida para **No Data**. O acionamento de **No Data** será acionado instantaneamente e não será levado **For** em consideração. Isso também pode resultar no não envio de uma notificação de OK se o alerta mudar de **No Data** -> **Pending** -> **OK**.

Se uma regra de alerta tiver um configurado **For** e a consulta violar o limite configurado, ela irá primeiro de **OK** para **Pending**. Indo de **OK** para **Pending** Grafana não enviará nenhuma notificação. Assim que a regra de alerta estiver disparando por mais **For** tempo, ela mudará para **Alerting** e enviará notificações de alerta.

Normalmente, é sempre uma boa ideia usar essa configuração, pois geralmente é pior obter um falso positivo do que esperar alguns minutos antes que a notificação de alerta seja acionada. Olhando para o **Alert list** ou **Alert list panels** você poderá ver os alertas em estado pendente.

Abaixo você pode ver um exemplo de linha do tempo de um alerta usando a **For** configuração. Em ~ 16: 04, o estado de alerta muda para **Pending** e após 4 minutos ele muda para **Alerting** quando as notificações de alerta são enviadas. Assim que a série volta ao normal, a regra de alerta volta para **OK**.



6.5.4. CONDIÇÕES

Atualmente, o único tipo de condição que existe é uma **Query** condição que permite especificar uma carta de consulta, intervalo de tempo e uma função de agregação.

Exemplo de condição de consulta

```
avg() OF query(A, 15m, now) IS BELOW 14
```

- `avg()` - Controla como os valores de **cada** série devem ser reduzidos a um valor que pode ser comparado ao limite. Clique na função para alterá-la para outra função de agregação.
- `query(A, 15m, now)` - A carta define qual consulta deve ser executada na guia **Métricas**. Os dois segundos parâmetros definem o intervalo de tempo, 15m, now significa 15 minutos atrás até agora. Você também pode 10m, now-2m definir um intervalo de tempo que será de 10 minutos a 2 minutos atrás. Isso é útil se você deseja ignorar os últimos 2 minutos de dados.

- IS BELOW 14 - Define o tipo de limite e o valor do limite. Você pode clicar em IS BELOW para alterar o tipo de limite.

Conditions

WHEN	avg ()	OF	query (A, 5m, now)	IS BELOW	14	
------	--------	----	--------------------	----------	----	--

+

If no data points or all values are null

SET STATE TO

No Data ▼

Test Rule

A consulta usada em uma regra de alerta não pode conter nenhuma variável de modelo. Atualmente apenas suportamos **AND** e **OR** operadores entre condições e eles são executados em série.

Por exemplo, temos 3 condições na seguinte ordem: *condição: A (avalia como: VERDADEIRO) OU condição: B (avalia como: FALSO) E condição: C (avalia como: VERDADEIRO)* então o resultado será calculado como ((VERDADEIRO OU FALSO) E VERDADEIRO) = VERDADEIRO.

Planejamos adicionar outros tipos de condição no futuro, como Other Alert, onde você pode incluir o estado de outro alerta em suas condições, e Time Of Day.

6.5.5. SÉRIE MÚLTIPLA

Se uma consulta retornar várias séries, a função de agregação e a verificação de limite serão avaliadas para cada série. O que Grafana não faz atualmente é rastrear o estado da regra de alerta **por série**. Isso tem implicações que são detalhadas no cenário abaixo.

- Condição de alerta com consulta que retorna 2 séries: **server1** e **server2**
- **server1** series faz com que a regra de alerta seja disparada e mudada para o estado **Alerting**
- As notificações são enviadas com a mensagem: *load peaking (server1)*
- Em uma avaliação subsequente da mesma regra de alerta, a série **server2** também faz com que a regra de alerta seja acionada
- Nenhuma nova notificação é enviada porque a regra de alerta já está em vigor **Alerting**.

Portanto, como você pode ver no cenário acima, o Grafana não enviará notificações quando outras séries fizerem com que o alerta seja disparado se a regra já estiver em estado **Alerting**. Para melhorar o suporte para consultas que retornam várias séries, planejamos rastrear o estado **por série** em uma versão futura.

Abaixo estão as condições em que você pode configurar como o mecanismo de avaliação de regras deve lidar com consultas que não retornam dados ou apenas valores nulos.

Sem opção de dados	Descrição
Sem dados	Defina o estado da regra de alerta para NoData
Alerta	Defina o estado da regra de alerta para Alerting
Manter o último estado	Mantenha o estado da regra de alerta atual, seja ele qual for.
OK	Não sei por que você deseja enviar um alerta quando as coisas estão bem, mas você pode.

Diga ao Grafana como lidar com erros de execução ou tempo limite.

Erro ou opção de tempo limite	Descrição
Alerta	Defina o estado da regra de alerta para Alerting
Manter o último estado	Mantenha o estado da regra de alerta atual, seja ele qual for.

Se você tiver um armazenamento de série temporal não confiável, do qual as consultas em algum momento expiram ou falham aleatoriamente, você pode definir esta opção Keep Last State para basicamente ignorá-las.

6.5.6. NOTIFICAÇÕES

Na guia de alerta, você também pode especificar notificações de regra de alerta junto com uma mensagem detalhada sobre a regra de alerta. A mensagem pode conter qualquer coisa, informações sobre como você pode resolver o problema, link para runbook e assim por diante.

As notificações reais são configuradas e compartilhadas entre vários alertas. Leia Notificações de alerta para obter informações sobre como configurar e definir notificações.

- **Enviar para** - Selecione um canal de notificação de alerta, se houver um configurado.
- **Mensagem** - digite uma mensagem de texto a ser enviada no canal de notificação. Alguns notificadores de alerta oferecem suporte à transformação do texto em HTML ou outros formatos avançados. Este campo oferece suporte a modelos .
- **Tags** - especifique uma lista de tags (chave / valor) a serem incluídas na notificação. Só é suportado por alguns notificadores .

As alterações do estado de alerta são registradas na tabela de anotação interna do banco de dados do Grafana. As mudanças de estado são visualizadas como anotações no painel gráfico da regra de alerta. Você também pode acessar o **State history** submenu na guia de alerta para visualizar e limpar o histórico de estado.

7. MODELO CAIXA

Conforme apresentado no tópico de arquitetura, o modelo de monitoramento CAIXA para as esteiras DevOps utiliza principalmente a arquitetura Prometheus/Grafana.

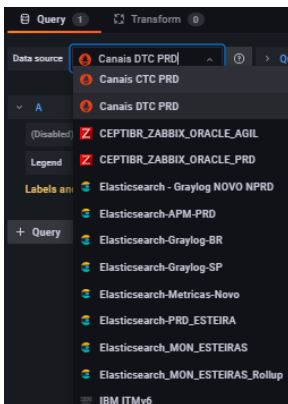
A instância corporativa do grafana está disponível em: <http://monitoracaodevops.produtos.caixa/>

Ainda que utilize principalmente o Prometheus, o modelo corporativo possui outras bases configuradas, como:

- elasticsearch (<https://www.elastic.co/>),
- Zabbix (<https://www.zabbix.com/>),
- influxdb (<https://www.influxdata.com/>), entre outras.

No modelo adotado pela CAIXA, o endpoint/metrics configurado trabalha na porta: 8778 (java) (jmx exporter).

A figura abaixo ilustra os principais data-sources disponíveis no grafana corporativo, com destaque para os canais CTC/DTC do prometheus.



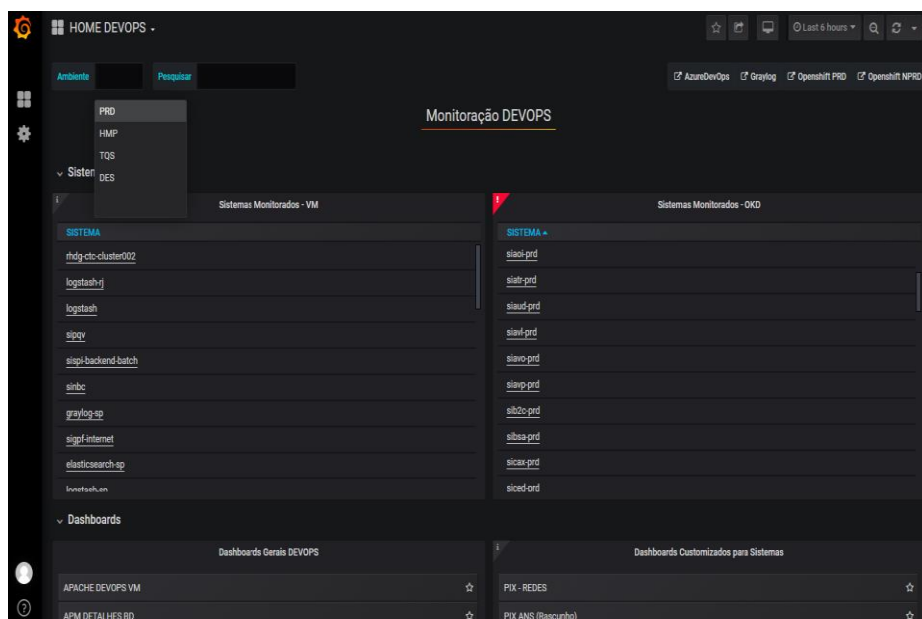
7.1. MONITORAÇÃO DEVOPS

O modelo de monitoração utilizado é o Monitoração DEVOPS, referente ao modelo de painel HOME DEVOPS que fica na pasta GENERAL (pasta raíz) e que serve como um “painel dos dashboards” e que apresenta duas principais variáveis: ambiente e pesquisa.

As opções da variável ambiente são DES, TQS, HMP e PRD

Na opção de pesquisa, o usuário deve incluir o nome do sistema que quer pesquisar. Se o sistema possuir um ou mais módulos publicados em VM, conforme a opção de ambiente escolhida, irá aparecer um link por módulo no painel “sistemas monitorados – VM”. Da mesma maneira, se tiver um ou mais módulos publicados no OKD, irá aparecer um link por módulo em “sistemas monitorados – OKD”. Os links servem para acessar os dashboards detalhados do sistema.

Além das variáveis, há quatro painéis para monitoração: (i) sistemas monitorados – VM, (ii) sistemas monitorados – OKD, (iii) painéis gerais e dashboards customizados para sistemas.



- **Sistemas monitorados – VM**

Este painel apresenta uma relação de sistemas que estão sendo monitorados e que usam ambientes de VM. O sistema acionado, via link, irá ser apresentado pelo modelo de painel INFRA DEVOPS VM

- **Sistemas monitorados – OKD**

Este painel apresenta uma relação de sistemas que estão sendo monitorados e que usam ambientes de OKD. O sistema acionado, via link, irá ser apresentado pelo modelo de painel INFRA DEVOPS OKD

- **Painéis Gerais**

Este painel apresenta uma relação de painéis disponíveis, com especial atenção para os painéis padrões INFRA DEVOPS VM e INFRA DEVOPS OKD.

Neste painel, é possível visualizar o sistema, usando variáveis, em modelos diferentes de INFRA DEVOPS VM e INFRA DEVOPS OKD.

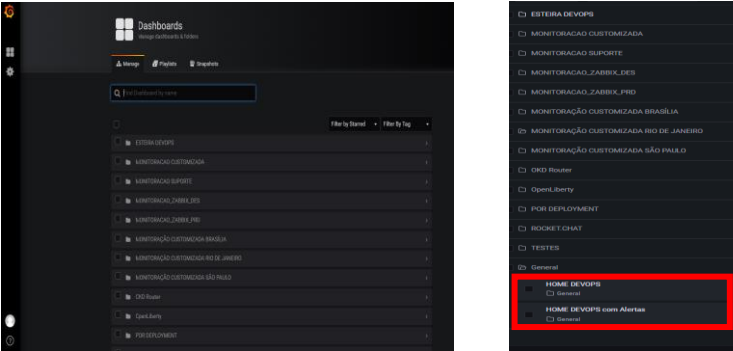
- **Dashboards customizados para sistemas**

Este painel apresenta uma relação de painéis customizados, solicitados e configurados por demanda.

7.2. ORGANIZAÇÃO DAS PASTAS

Na opções de gerenciamento de painéis, é possível visualizar os diversos painéis disponíveis e suas tags de identificação organizados em pastas.

A figura abaixo apresenta a organização de pastas, incluindo as principais, ESTEIRA DEVOPS e a *GENERAL*, onde se situa a painel padrão de visualização, a ESTEIRA DEVOPS.



7.3. PRINCIPAIS PAINÉIS

Além do painel ESTEIRA DEVOPS, dois modelos de dashboards são referências para o atual modelo adotado em <http://monitoracaodevops.produtos.caixa/>: INFRA DEVOPS VM e INFRA DEVOPS OKD

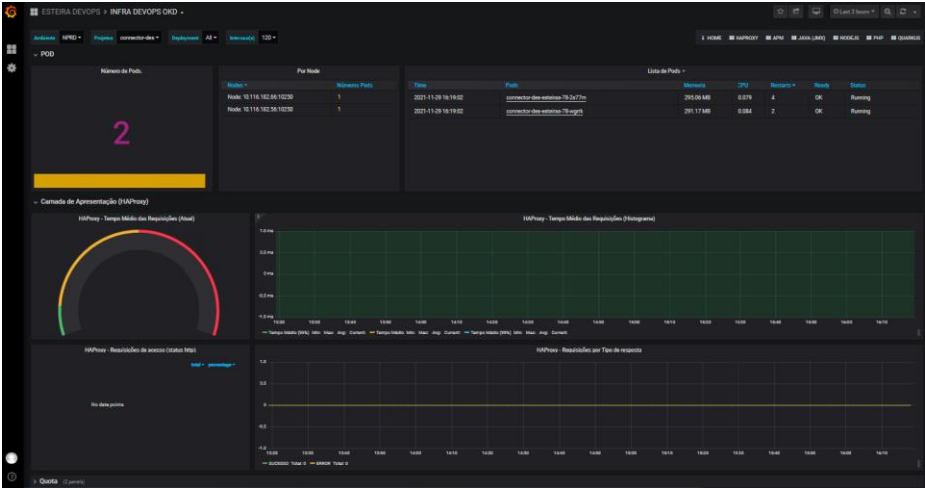
7.3.1. INFRA DEVOPS OKD

O painel INFRA DEVOPS OKD é um painel rico em visões, que apresenta as seguintes variáveis: datasource (prometheus), node (vários label_values), projeto (vários label_values), deployment (vários label_values) e Pods (vários label_values), além de um conjunto de intervalos.

Variable	Definition
datasource	prometheus
Node	label_values(node)
Host_ip	
projeto	label_values(kube_namespace_labels(label_createdby="ads"), namespace)
Deployment	label_values(kube_endpoint_info(namespace=~"\$projeto"), endpoint)
Pods	label_values(kube_pod_container_info(namespace=~"\$projeto"), pod)
nomeProj	
interval	1,5,10,15,30,45,60,120,300,600,900,1800

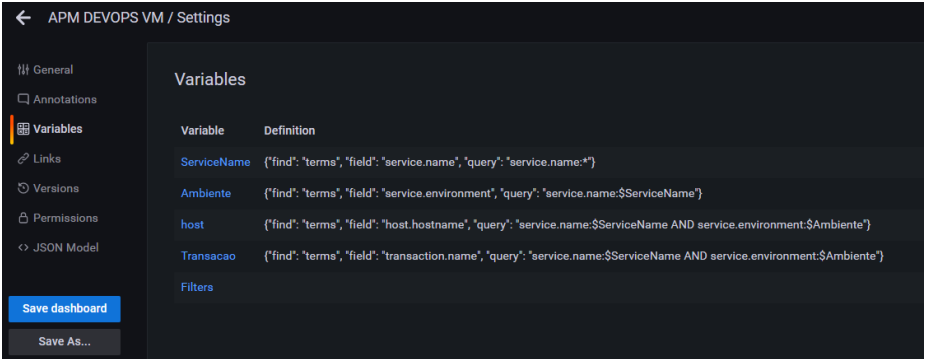
A imagem abaixo apresenta a interface do painel INFRA DEVOPS OKD com as variáveis situadas no lado esquerdo e acima e links para outros tipos de painel no lado direito.

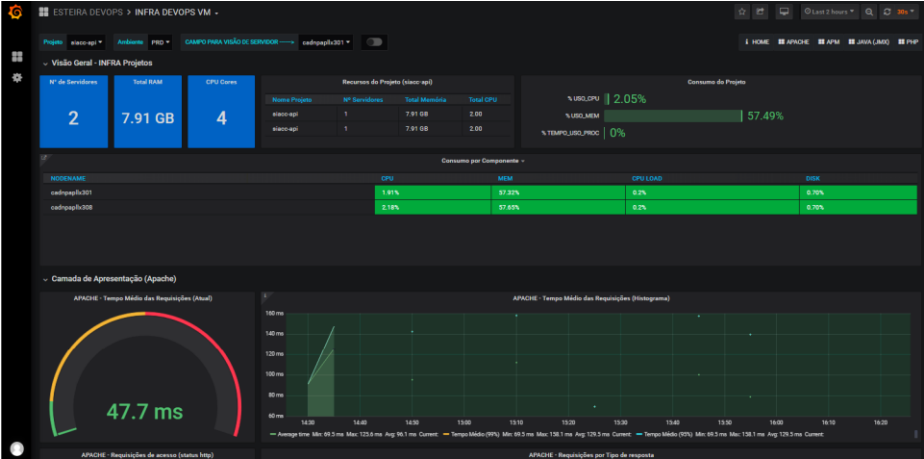
Abaixo das variáveis, segue um conjunto de painéis suficientes para monitorar toda a infra em uso, como o número de Pods, a lista de Pods, tempo médio de requisições em formato gauge e histograma, entre outros.



7.3.2. INFRA DEVOPS VM

O painel INFRA DEVOPS VM apresenta uma estrutura parecida, mas apresenta algumas diferentes variáveis e painéis, adequada a monitoração da tecnologia de VM.





As variáveis projeto, ambiente e servidor podem ser vistos no menu da interface do painel INFRA DEVOPS OKD.

8. SUPORTE

Atualmente, o suporte ao Grafana/Prometheus é mantido pela CETAD.

Para implementação, atualização, inovação, ajustes e adequação de infra (CETAD36) para o grafana (<http://monitoracaodevops.produtos.caixa/>) e os data sources de vinculação, a unidade responsável é a CETAD36.

Para suporte na monitoração, criação de painéis, visões e consultas, a unidade responsável é a CETAD10.

A solicitação desses serviços devem ser criados via servicos.caixa.

Caminho -> Tecnologia da Informação e Comunicação / Centralizadoras da Tecnologia da Informação / CETAD / CETAD Gerenciamento (vide imagens)

The screenshot shows the 'Solicitar Serviço' (Request Service) interface. On the left, there's a sidebar with a tree view of services. The selected service is 'CETAD - Gerenciamento'. The main form area contains several fields for providing details about the service request, including a link to the monitoring panel, the name of the environment, the type of operation, the priority level, and the link to access the service. There are also buttons for 'Salvar como Rascunho' (Save as Draft) and 'Enviar' (Send).

Além da escolha dos ambientes, deve-se optar pelo serviço (criação, alteração ou exclusão), informar o link do painel (exceto criação), a criticidade (criação) e a descrição do serviço desejado.